Reviewer Pack — Schur-Weyl Component Count Gap in Monotone Circuit Size for …

Entry e003278ab5b9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 09:19:31 UTC

Schur-Weyl Component Count Gap in Monotone Circuit Size for Permanent vs Determinant

Entry ID: `e003278ab5b9` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-13 09:19:31 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Monotone Circuit Size for Permanent

Statement:

> For n ≥ 2, the number of irreducible components in the Schur-Weyl decomposition of the permanent polynomial perm_n is exponentially larger than that of the determinant det_n, specifically, the ratio of their component counts is Ω(2^{n/2}).

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes tensor products into irreducible representations, and the permanent's decomposition under symmetric group actions may inherently require more components due to its non-symmetric nature, suggesting a structural complexity gap that could imply lower bounds on monotone circuit size.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7538fc02bac6018a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM₁	LLM₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from itertools import combinations, permutations

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(n):
    count = 0
    for partition in combinations(range(1, n), n - 1):
        hook_length = 1
        for i in range(n):
            hook_length *= (n - i) * (n - partition[i]) // (i + 1)
        count += factorial(n) // hook_length
    return count

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    perm_components = hook_length_formula(n)
    det_components = hook_length_formula(n)
    ratio = perm_components / det_components
    conjecture_holds = ratio >= 2**(n/2)
    counterexample = "" if conjecture_holds else f"Ratio {ratio} < 2^{n/2}"
    return {
        "metric_name": "component_ratio",
        "metric_value": ratio,
        "instances_tested": n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]
```

```
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean_ratio = sum(r['metric_value'] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)
    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_ratio} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conj
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)]['cour
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673a101a.py", line 54, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673a101a.py", line 37, in run_trial
    perm_components = hook_length_formula(n)
                      ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673a101a.py", line 30, in hook_length_f
    hook_length *= (n - i) * (n - partition[i]) // (i + 1)
                                  ~~~~~~~~~^^^
IndexError: tuple index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with IndexError in hook_length_formula, preventing component count calculation | next: Fix partition indexing in hook_length_formula implementation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	110941
2	propose	ollama_remote	qwen3:8b	0	0	80224
3	preregistration	ollama_remote	qwen3:8b	0	0	24519
4	novelty	ollama_remote	qwen3:8b	0	0	20633
5	novelty	ollama_remote	qwen3:8b	0	0	12850
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10977
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7335
8	judge	ollama_remote	qwen3:8b	0	0	16790

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 284269 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/e003278ab5b9.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e003278ab5b9.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e003278ab5b9.tar.gz` (if generated)